Guide to the Lab Sessions

Reinforcement Learning and Generative AI

University of Alcalá

October 27, 2026

Contents

1 Introduction **2**

2 Course Structure **2**

3 What is a Jupyter Notebook (`.ipynb`)? **2**

4 Accessing the Notebooks (Blackboard & GitHub) **2**

5 Opening a Notebook: Two Options **3**
5.1 Option A: Google Colab (recommended) 3
5.2 Option B: Working Locally with an IDE 3

6 Working on the Notebook **4**

7 Saving Your Work **5**

8 Class Competition: Automated GitHub Leaderboard **5**

9 Tournament Scoring: Point Formulas **6**
9.1 LunarLander-v2 / v3 6
9.2 Taxi-v3 / v4 7

10 Head-to-Head Agent Duel: Comparing Against Classmates **7**

11 Submitting Your Lab Assignment **8**

12 Evaluation Overview **8**

13 Getting Help **9**

1 Introduction

This document explains how the lab sessions of the Reinforcement Learning (RL) module of the **Reinforcement Learning and Generative AI** course are organized, and how to work with the provided materials. These materials are distributed as **Jupyter Notebooks** (`.ipynb` files). The purpose of this guide is to help every student get started, even without any prior experience using notebooks or Google Colab.

2 Course Structure

The course combines theory sessions (9h) and lab sessions (6h). The lab component consists of **three mandatory deliverables**, each based on a Jupyter Notebook:

Session	Topic	Notebook
Lab 1	Introduction to RL & Deep PPO	`Lab1/Lab1.ipynb`
Lab 2	Tabular Q-Learning (Taxi-v3/v4)	`Lab2/Lab2.ipynb`
Final Lab	Deep Q-Learning with Atari	`FinalProject/Final_Lab.ipynb`

In every lab session, students will: interact with RL environments, train models following a guided (Colab-based) tutorial, implement the requested parts of the code, and submit their final solution. **All three deliverables** (Lab 1, Lab 2, and Final Lab) are evaluated and contribute to the **Lab Mark (LM)** of the course. Together with a short oral test on the labs, they account for **60%** of the RL module grade. The remaining **40%** corresponds to a written exam on the theory of the module.

3 What is a Jupyter Notebook (`.ipynb`)?

A Jupyter Notebook is a document that mixes executable code (in this course, Python) with explanatory text, images, and the results of execution, all in the same file. It is organized into *cells*, of two main types:

- **Code cells**: contain Python code that can be executed by clicking the Play button or pressing `Shift + Enter`.
- **Text cells (Markdown)**: contain explanations, instructions, equations and images, but no executable code.

You do **not** need to install Python or any library on your own computer to work with these notebooks: the recommended option is **Google Colab**, a free cloud platform provided by Google that runs notebooks directly in your web browser. However, if you prefer to work **locally** on your own computer using an IDE (e.g. VS Code, PyCharm or Jupyter itself), this is also possible and explained in this guide.

4 Accessing the Notebooks (Blackboard & GitHub)

The `.ipynb` files for the lab sessions **are available both on Blackboard** (the University of Alcalá's e-learning platform) and directly in the official **course GitHub repository** (https://github.com/gramuah/RL_tournament).

Step by step

1. Navigate to the course page on Blackboard and locate the section for the corresponding lab session (Lab 1, Lab 2, or Final Lab).
2. Click on the `.ipynb` file to download it (e.g. `unit1_uah_student.ipynb`). Depending on your browser, it may download directly or open a preview with a download button.
3. Save the file somewhere easy to find on your computer, such as the *Downloads* folder or Desktop.

Important

Do **not** try to open the `.ipynb` file by double-clicking it directly on your computer: it will not display correctly, since it requires a Jupyter/Colab environment to be rendered and executed. Follow the steps in the next section instead.

5 Opening a Notebook: Two Options

Once you have the `.ipynb` file downloaded from Blackboard, you can choose one of the following two options to work with it. **Option A (Google Colab)** is recommended for most students, since it requires no installation and runs in the cloud. **Option B (Local IDE)** is for students who prefer to work on their own computer, already have Python installed, or want full control over their environment.

5.1 Option A: Google Colab (recommended)

Step by step

1. Go to `https://colab.research.google.com`.
2. Sign in with a Google account (any personal Gmail account works).
3. In the welcome window, click on the tab **Upload**. If it does not appear, go to **File** → **Upload notebook**.
4. Click **Browse** (or drag and drop the file) and select the `.ipynb` file you downloaded from Blackboard.
5. The notebook will open directly in Colab, ready to be executed.

Advantages of this option: no installation required, free access to GPUs, and your work can be saved directly in your Google Drive (see Section 7).

5.2 Option B: Working Locally with an IDE

If you prefer to run the notebook on your own machine, you will need to install Python and a few additional tools. This option is suitable if you already have a Python environment set up, or if you want to avoid depending on an internet connection.

Step by step

1. **Install Python** (version 3.9 or higher is recommended) from https://www.python.org/downloads/, if not already installed.
2. **Install an IDE with notebook support**, for example:
   - **VS Code**: https://code.visualstudio.com/, plus the official *Python* and *Jupyter* extensions (installable from the Extensions tab inside VS Code).
   - **PyCharm Professional**: https://www.jetbrains.com/pycharm/, which has built-in Jupyter notebook support.
   - Alternatively, install **Jupyter Notebook/JupyterLab** directly: open a terminal and run `pip install notebook`, then launch it with `jupyter notebook`.
3. **Open the `.ipynb` file** you downloaded from Blackboard using your chosen IDE (e.g. **File** → **Open** in VS Code/PyCharm, or navigating to it from the Jupyter file browser).
4. **Select a Python interpreter/kernel** for the notebook (VS Code and PyCharm will prompt you to choose one the first time you open a notebook).
5. **Install the required libraries**. Most notebooks include a first code cell with `!pip install ...` commands; run that cell first. If needed, you can also install packages manually from a terminal with `pip install` *`package-name`*.

Important

When working locally, your computer's hardware (CPU/GPU) will be used for training, which may be significantly slower than Colab's free GPUs if your machine does not have a dedicated graphics card. It is also your responsibility to manage library versions and dependencies correctly. If you encounter installation issues, Option A (Google Colab) remains available as a fallback.

Note for Windows and macOS users regarding virtual displays

The lab notebooks automatically detect your platform. Linux tools like `apt-get` and virtual displays (`Xvfb`) are only executed in headless cloud environments like Google Colab. On Windows and macOS, the notebook automatically skips `apt-get` and `Xvfb`, using your operating system's native graphics subsystem directly.

6 Working on the Notebook

Once the notebook is open, either in Colab or in your local IDE, keep the following in mind:

- **Run cells in order**, from top to bottom, using the Play button to the left of each cell (or `Shift + Enter`). Skipping cells or running them out of order can cause errors.
- **Read the text cells carefully**: they contain the explanations and instructions needed to understand and complete the exercises.
- **Complete the sections marked for you**, usually indicated with comments such as `# TODO` or similar placeholders, following the given instructions.
- **(Colab only) Enable a GPU** when training models, since it speeds up computation considerably: go to **Runtime** → **Change runtime type** → **Hardware accelerator** →

GPU, then save. If working locally, make sure your own GPU drivers (e.g. CUDA) are correctly configured, or expect slower training on CPU.

- If a cell throws an error, re-read the previous cells: you may be missing a required installation step (e.g. `!pip install ...`) or a variable defined earlier that was not executed.

7 Saving Your Work

If you are using **Google Colab**, it **does auto-save your code changes** to Google Drive (once you have saved a copy there), so you generally do not lose your code between sessions. However, there are two important caveats:

1. **First, save a copy to your Drive.** As soon as you open the notebook, go to **File** → **Save a copy in Drive**. This creates your own editable copy in *My Drive* → *Colab Notebooks*. From this point on, Colab auto-saves your code.
2. **Cell outputs are NOT auto-saved.** When you close a Colab session, the computed outputs (training logs, graphs, videos, printed results) are *not* stored by default. If you re-open the notebook without re-running the cells, the outputs will be blank. To avoid losing outputs for submission:
   - Go to **Edit** → **Notebook settings** and make sure "Omit code cell output when saving notebook" is **unchecked**.
   - Before downloading for submission, go to **Runtime** → **Run all** to re-execute all cells and restore their outputs.
   - Then download with **File** → **Download** → **Download .ipynb** to get the notebook *with all outputs included*.
3. You can reopen your copy anytime from `https://colab.research.google.com` under the **Recent** tab, or directly from Google Drive.

If you are working **locally** with an IDE, simply save the file regularly using the standard shortcut (`Ctrl+S`), as you would with any other document, and keep a backup copy of your work (e.g. in a personal folder or cloud storage) in case of hardware issues.

8 Class Competition: Automated GitHub Leaderboard

In **Lab 1** and **Lab 2**, the final section of the notebook will guide you through the packaging of your agent: an `agent.py` file (which defines an `agent(observation)` function) together with your trained model weights (`ppo-LunarLander-v3.zip` or `q-learning.pkl`).

To foster friendly competition, peer learning, and benchmark reproducibility, all agents participate in an **automated class tournament hosted directly on GitHub**:

1. **Submission via GitHub Pull Request**: The notebook includes a 1-click cell that automatically forks the repository, uploads your agent files (`agent.py` and your trained model), and opens a Pull Request on your behalf using your GitHub Personal Access Token (PAT). Alternatively, you can do it manually by placing your files in:
   - Lab 1: `submissions/lab1/<your_github_username>/`
   - Lab 2: `submissions/lab2/<your_github_username>/`

and opening a **Pull Request (PR)** against the `main` branch of the repository.

How to get a GitHub PAT

(a) Go to https://github.com/settings/tokens (log in to GitHub first).

(b) Click **"Generate new token (classic)"**.

(c) Choose a name (e.g. `rl-tournament-lab`), set expiration to at least 30 days, and tick the `repo` checkbox.

(d) Click **"Generate token"** and **copy the token** immediately (starts with `ghp_...`). You cannot retrieve it again after leaving the page.

2. **Instant CI Evaluation**: Once your PR is open, a GitHub Actions bot automatically evaluates your agent for **30 episodes** on fixed, reproducible seeds and posts a detailed evaluation report (mean reward, standard deviation, success rate) as a comment on your PR within approximately 2 minutes. No action required on your part.

3. **Official Leaderboard**: Once an instructor reviews and merges your PR, your score appears on the official tournament website at https://gramuah.github.io/RL_tournament/, updated automatically.

Leaderboard Tip

You can submit multiple times! If you improve your hyperparameters, refine reward engineering, or discover a better architecture, open an updated PR to improve your standing on the leaderboard.

9 Tournament Scoring: Point Formulas

The score on the leaderboard and in duels is the mean cumulative reward obtained by the agent over the evaluation episodes. Points are calculated according to the following formulas:

9.1 LunarLander-v2 / v3

$$R_{\text{LunarLander}} = R_{\text{trajectory}} + 10 \cdot N_{\text{legs}} - 0.3 \cdot N_{\text{main}} - 0.03 \cdot N_{\text{side}} + R_{\text{terminal}}$$

where:

- $R_{\text{trajectory}} \in [100, 140]$: Points awarded for moving from the top of the screen to the landing pad (rewarding low distance, low velocity, and upright orientation).
- $N_{\text{legs}} \in \{0, 1, 2\}$: $+10$ points for each leg making contact with the lunar surface.
- N_{main}: Total frames firing the main thruster (-0.3 points per frame).
- N_{side}: Total frames firing lateral thrusters (-0.03 points per frame).
- R_{terminal}: $+100$ points for a safe touchdown inside the helipads, or -100 points for a crash.

9.2 Taxi-v3 / v4

$$R_{\text{Taxi}} = 20 \cdot \mathbf{1}_{\{\text{delivered}\}} - 1 \cdot (\text{steps}) - 10 \cdot (\text{illegal_actions})$$

where:

- $+20$ points for successfully delivering the passenger at the destination.
- -1 point for each step taken (penalizing long trajectories).
- -10 points for each illegal pickup or drop-off attempt.

10 Head-to-Head Agent Duel: Comparing Against Classmates

Beyond overall leaderboard ranks, students can directly challenge and compare their agents against baselines or other classmates in a **Head-to-Head Duel**:

- **Identical Benchmark Seeds**: Both agents face the exact same initial state, landing terrain, wind turbulence (in LunarLander), or passenger pickup/drop-off locations (in Taxi).
- **Speed vs. Score Analysis**: Discover who lands or delivers faster (fewer time steps), who consumes less fuel/actions, and who achieves higher reward stability.
- **Side-by-Side Synchronized Video**: Generates a broadcast-style split-screen video with a real-time HUD (speed, step counter, accumulated reward, touchdown status) and celebration victory screens.
- **Challenge Any Classmate**: By entering the GitHub username of any student on the leaderboard, the comparator tool automatically pulls their model from the public repository and runs the duel against your local model.

How to View and Run Duels Directly on GitHub

You do not need to install anything locally or run Python to visualize duels:

1. **Interactive Web Arena (GitHub Pages)**: Visit the official public web arena at `https://gramuah.github.io/RL_tournament/`. You can pick any two participants from dropdown menus, click **"Start Duel"**, and watch the split-screen replay with live landing speed gauges, fuel burn rates, and comparative statistics.
2. **Animated GIFs in `LEADERBOARD.md`**:
   The repository's official leaderboard (see `LEADERBOARD.md`) includes embedded animated duel replays showing the agents maneuvering and landing side-by-side directly inside GitHub Markdown.
3. **On-Demand Duels via GitHub Actions**: In the **Actions** tab of the repository (see `Actions → duel.yml`), click **"Run workflow"**, select your agent and an opponent, and the CI bot will execute the match and post an animated GIF replay with full scorecards directly in the job summary.
4. **Inside Google Colab**: Run the dedicated duel cell provided at the end of each lab notebook to execute duels and render the synchronized video inline.

11 Submitting Your Lab Assignment

At the end of each lab session (or by the deadline indicated by the instructor), you must submit your work for evaluation. To do so:

1. Make sure **all cells have been executed** and their outputs are visible. Do not clear the outputs before submitting: the instructor needs to see the results of your training/experiments.
2. Download/export your notebook as an `.ipynb` file:
   - **Google Colab**: **File** → **Download** → **Download .ipynb**.
   - **Local IDE**: the file on your disk *is* already the `.ipynb` file; just make sure it has been saved with the latest changes (`Ctrl+S`).
3. Rename the file following the convention:
   `Surname_Name_LabX.ipynb`
   (e.g. `Garcia_Juan_Lab1.ipynb`).
4. **For Lab 1 and Lab 2**, also download the `submission.zip` file generated at the end of the notebook (see Section 8).
5. Submit both the notebook and, when applicable, the `submission.zip` file through the **Blackboard** assignment/submission box corresponding to that lab session, before the deadline indicated by the instructor.

Important

All three deliverables (**Lab 1**, **Lab 2** and the **Final Lab**) are mandatory and must be submitted through **Blackboard**. Submission to the GitHub tournament (Pull Request) is **recommended** but **not mandatory**: if you do not submit to GitHub but your completed notebook is submitted on Blackboard, your grade will not be penalized. However, participating in the tournament allows your agent to be automatically evaluated and ranked against your classmates, and it is a great learning experience. Late or missing Blackboard submissions may negatively affect your final mark.

12 Evaluation Overview

As a reminder, the final mark of the course is composed as follows:

- **Lab Mark (60% of RL module grade)**: based on all three deliverables — **Lab 1**, **Lab 2**, and the **Final Lab** — plus a short oral test on the lab work.
- **Written Exam (40% of RL module grade)**: covers the theoretical content of the RL module.

Note on subject name

These labs belong to the **Reinforcement Learning and Generative AI** subject. The Lab Guide and notebooks cover the Reinforcement Learning component. Generative AI content is addressed separately within the same subject.

Originality and Research Bonus

In all lab assignments, originality, critical thinking, and independent research are actively valued and rewarded. Students who go beyond the baseline solutions—by thoroughly experimenting with hyperparameters, exploring alternative algorithmic strategies or neural network architectures, and clearly documenting their investigations—will be rewarded with higher marks.

13 Getting Help

If you encounter errors or have doubts while working on a notebook:

- Re-read the instructions in the notebook: most exercises include hints or links to relevant documentation.
- For Lab 1, consult the **Stable-Baselines3 official documentation**: https://stable-baselines3.readthedocs.io/en/master/. The PPO hyperparameter reference is at https://stable-baselines3.readthedocs.io/en/master/modules/ppo.html.
- If you need a **Hugging Face token** (for uploading your model to the Hub in Section 8 of Lab 1), go to https://huggingface.co/settings/tokens, create a free account if needed, and generate a token with **Write** role. Keep it private and do not share it.
- Check the original Hugging Face Deep RL Course material on which these labs are based:
  - Unit 1: https://huggingface.co/learn/deep-rl-course/en/unit1/hands-on
  - Unit 2: https://huggingface.co/learn/deep-rl-course/en/unit2/hands-on
  - Unit 3: https://huggingface.co/learn/deep-rl-course/en/unit3/hands-on
- Ask the instructor during the lab sessions, or by email, if the problem persists.